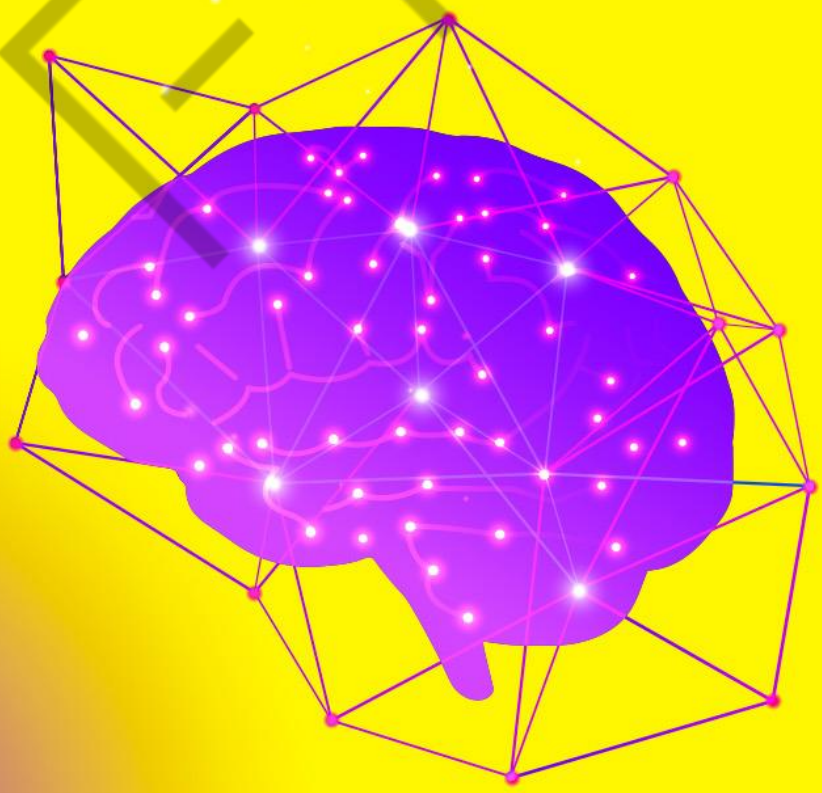
BUILDING STATISTICAL MODELS

REDES NEURAIS CONVOLUCIONAIS (CNNS)



9

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Analogia do funcionamento de uma CNN comparada ao córtex visual biológico 6
Figura 2 – Representação de uma imagem colorida em RGB 8
Figura 3 – Representação de uma imagem tons de cinza 8
Figura 4 10
Figura 5 11
Figura 6 12
Figura 7 13
Figura 8 14
Figura 9 14
Figura 10 15
Figura 11 17
Figura 12 18
Figura 13 20
Figura 14 21
Figura 15 22
Figura 16 27
Figura 17 28
Figura 18 29
Figura 19 30
Figura 20 32
Figura 21 34
Figura 22 35
Figura 23 37
Figura 24 37
Figura 25 38
Figura 26– Saída do Código-fonte 9 Fonte: Elaborado pelo autor (2024) 40
Figura 27– Saída do Código-fonte 10 Fonte: Elaborado pelo autor (2024) 41
Figura 28 43
Figura 29 46

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE TABELAS

Tabela 1 – Funções de ativação para CNNs.....19

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Imports do código de CNN.....32
Código-fonte 2– Carregando o dataset 33
Código-fonte 3 – Chegando o tamanho dos conjuntos de treino e teste.....33
Código-fonte 4– Pré-processamento dos dados 34
Código-fonte 5– Treinamento do modelo baseline, criado à mão 36
Código-fonte 6– Calculando a acurácia do modelo inicial.....37
Código-fonte 7– Plotando a matriz de confusão do modelo inicial.....37
Código-fonte 8– Pré-processamento dos dados para uso da VGG16 para transfer learning 39
Código-fonte 9– Observando as camadas da VGG16 39
Código-fonte 10 – Completando o modelo de transfer com camadas de classificação 40
Código-fonte 11– Compilando o treinando o modelo resultante.....42
Código-fonte 12– Avaliação do modelo com transfer learning 42

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

REDES NEURAIS CONVOLUCIONAIS (CNNS) 6
1 INTRODUÇÃO 6
1.1 Importância e aplicações de CNNs 6
1.2 Representação de uma imagem 7
1.1.1 Extração de features de imagens 9
2 ARQUITETURAS CONVOLUCIONAIS 10
2.1 Estrutura básica 10
2.2 Entendendo operações de Convolução e Pooling.....12
2.3 Configurações avançadas 16
2.3.1 Padding e Striding 16
2.3.2 Funções de ativação 18
2.3 Terminologia e evolução 19
3 TREINAMENTO DE REDES NEURAIS CONVOLUCIONAIS.....23
3.1 Preparação de dados 24
3.2 Técnicas de regularização.....24
3.3 Otimização de hiperparâmetros 25
3.4 Operações sobre volumes.....26
4 DESAFIOS E AVANÇOS EM REDES NEURAIS CONVOLUCIONAIS 27
4.1 Técnicas de ampliação da base de dados 28
4.2 Transfer Learning e Fine-tuning 29
5 MÃO NA MASSA! DEEP LEARNING EM PYTHON 31
6 HANDS ON: PRATICANDO O USO DE CNNS 43
6.1 Aquecimento 43
6.2 Seu desafio 44
6.3 Proposta de solução.....45
REFERÊNCIAS.....47
GLOSSÁRIO 48

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REDES NEURAIS CONVOLUCIONAIS (CNNS)

1 INTRODUÇÃO

1.1 Importância e aplicações de CNNs

As Redes Neurais Convolucionais (do Inglês "*Convolutional neural network*" – CNNs) representam uma classe poderosa de algoritmos de aprendizado profundo que revolucionaram a área de visão computacional e além.

Inspiradas na organização do córtex visual biológico, as CNNs são capazes de aprender padrões complexos em dados de entrada, especialmente em imagens, tornando-se indispensáveis em uma ampla gama de aplicações. Sua arquitetura única permite a extração automatizada de características relevantes em imagens, tornando-as ideais para tarefas como reconhecimento de objetos, classificação de imagens, detecção de objetos, segmentação semântica e muito mais. Sua lógica de funcionamento está representada de forma simplificada na Figura 1:

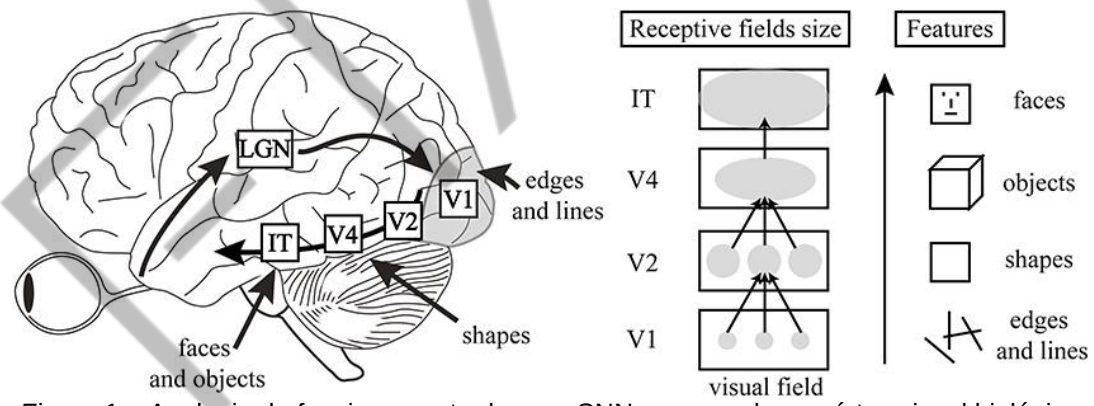


Figura 1 – Analogia do funcionamento de uma CNN comparada ao córtex visual biológico
Fonte: Imagem reproduzida de: https://gracewlindsay.com/2018/05/17/deep-convolutional-neural-networks-as-models-of-the-visual-system-qa/

O córtex visual biológico é como um computador no cérebro, que processa o que vemos. Ele divide as imagens em pequenas partes e as analisa para reconhecer cantos, linhas, formas, cores e objetos. Depois, junta essas informações para formar uma imagem completa, permitindo que enxerguemos o mundo ao nosso redor. De maneira similar, as features de uma imagem são extraídas de forma hierárquica em

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

uma CNN, compreendendo o conteúdo daquela imagem aos poucos. É como um quebra-cabeça sendo montado peça a peça.

Essa capacidade de aprender características hierárquicas e invariantes a translações, rotações e escalas destacam as CNNs pela sua eficiência e precisão na análise de dados visuais, desempenhando um papel fundamental em inúmeras áreas, incluindo medicina, veículos autônomos, segurança, entretenimento e tantas outras.

1.2 Representação de uma imagem

Antes de prosseguirmos, é importante estarmos familiarizados com a representação computacional de uma imagem. Afinal, para um computador, o que é uma imagem?

Quando uma imagem é representada em um computador, ela é dividida em **pixels**, que são os pequenos pontos coloridos que formam a imagem.

Existem diversos espaços de características para representar os pixels. O mais comum é o **RGB**, no qual a imagem é representada por uma matriz tridimensional, onde as duas primeiras dimensões correspondem à **altura** e **largura** da imagem, e a terceira dimensão representa os três **canais de cores**: vermelho (R – *red*), verde (G – *green*) e azul (B – *blue*).

Cada canal representa a **intensidade de cor em uma escala de 0 a 255**:

- O valor (255, 0, 0) representa o vermelhão puro;
- O valor (0, 255, 0) representa o verde puro;
- O valor (0, 0, 255) representa o azul puro.

A cor do pixel que enxergamos é a soma das representações dos três canais. Por exemplo, uma imagem de dimensões 100x100 pixels em RGB seria representada por uma matriz de tamanho 100x100x3. Cada elemento dessa matriz contém três valores, um para cada canal de cor, representando a intensidade de vermelho, verde e azul do pixel correspondente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

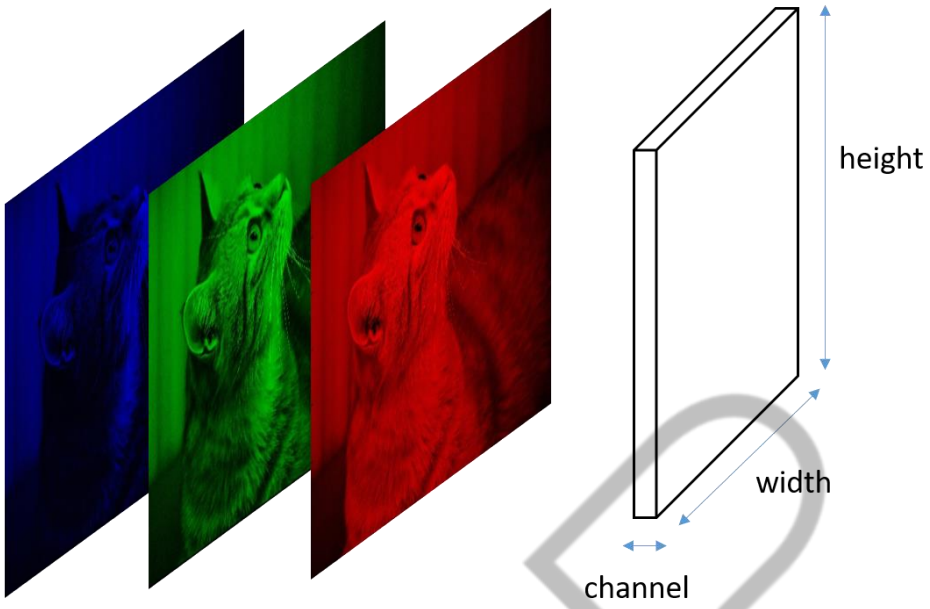


Figura 2 – Representação de uma imagem colorida em RGB
Fonte: Imagem reproduzida de: https://towardsdatascience.com/understanding-images-with-skimage-python-b94d210afd23

Para imagens em escala de cinza, há apenas um canal de cor, onde cada pixel é representado por um único valor de intensidade de cinza, variando de 0 (preto) a 255 (branco). Nesse caso, quanto maior o valor do pixel, mais claro ele é, e quanto menor, mais escuro.

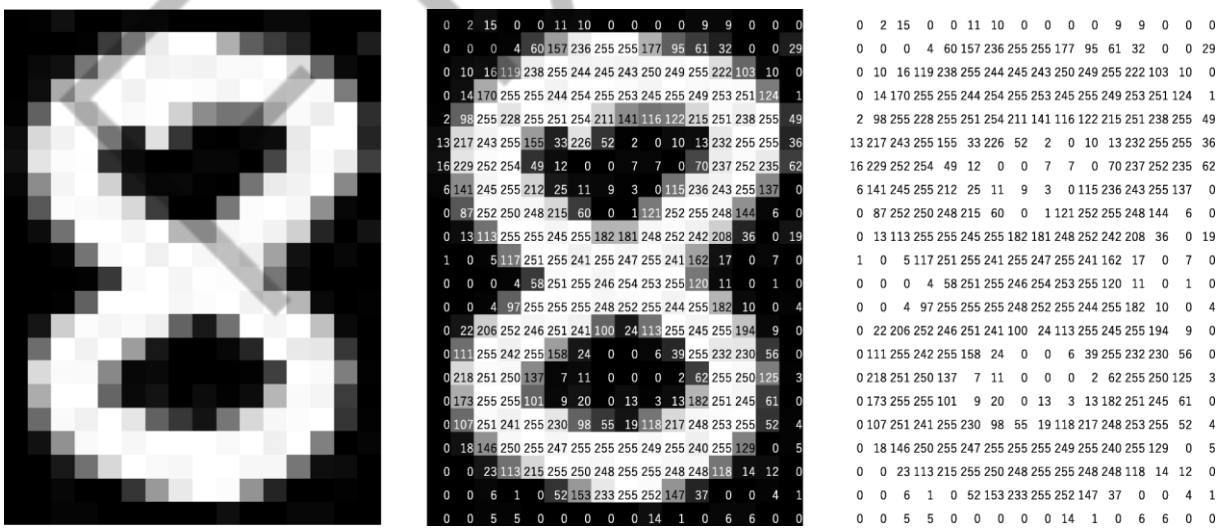


Figura 3 – Representação de uma imagem tons de cinza
Fonte: Imagem reproduzida de: https://medium.com/lifeandtech/convert-csv-file-to-images-309b6fdb8c49

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A Figura 3 mostra uma imagem em escala de cinza, representando o número “8” escrito à mão. Note que cada linha e coluna representa um pixel, expresso de 0 a 255. Os valores próximos a 0 representam o mais escuro possível, e os valores próximos a 255, o mais claro possível.

Essa representação de 0 a 255 é uma forma de normalização dos valores dos pixels, onde 0 representa a ausência de cor e 255 representa a cor mais intensa. Isso facilita o processamento da imagem pelo computador e a manipulação dos valores de cor para realizar tarefas como detecção de bordas, reconhecimento de objetos e classificação de imagens.

1.1.1 Extração de features de imagens

Todo modelo preditivo trabalha com features! Features são as características que descrevem os dados. No caso de dados estruturados, cada coluna costuma ser uma feature daquele exemplo (daquela linha). No caso de imagens, precisamos extrair features diretamente dos pixels, porque usá-los diretamente para alimentar modelos preditivos pode ser muito limitador.

Na visão computacional clássica, a extração de features em imagens frequentemente envolvia a criação de descritores feitos à mão, projetados especificamente para tarefas individuais, como detecção de bordas, ou regiões de interesse, chamadas de *blobs*. Esses descritores eram construídos com base em conhecimento prévio sobre o problema e eram muitas vezes adaptados para cada aplicação específica. Por exemplo, para detectar bordas em uma imagem, poderíamos usar o operador Sobel.

Com o tempo, surgiram descritores mais gerais, capturando características como cor, textura e forma, sendo úteis para uma variedade de problemas de visão computacional. Exemplos incluem histogramas de cores, matrizes de coocorrência de tons de cinza e descritores baseados em transformada de Fourier para textura.

Além disso, surgiram algoritmos de detecção e descrição de características, como SIFT (“*Scale-Invariant Feature Transform*” – Transformada de Features Invariante à Escala) e SURF (“*Speeded Up Robust Features*” – Features Robustos e Acelerados), que buscavam características invariantes à rotação, escala e iluminação

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

em imagens. Esses descritores eram mais robustos em relação a transformações geométricas e mudanças na iluminação. Note, na Figura 4, que as mesmas características do objeto foram encontradas independentemente da variação de orientação e posicionamento da imagem.

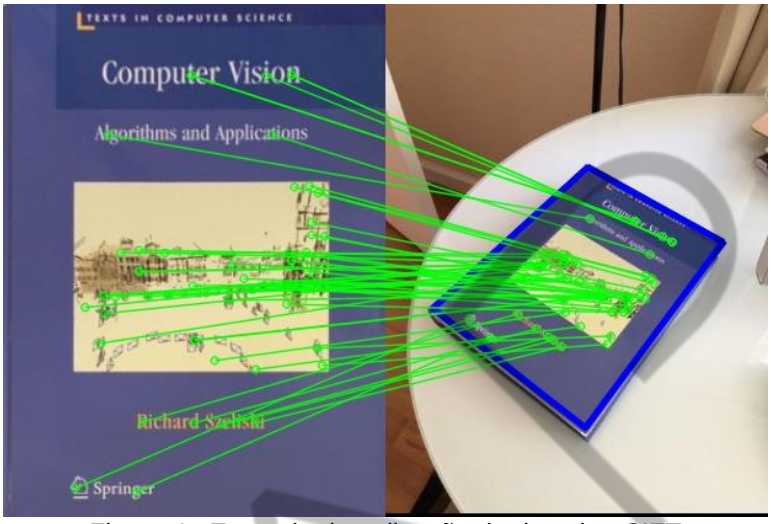


Figura 4– Exemplo de aplicação do descritor SIFT
Fonte: Imagem reproduzida de: https://stackoverflow.com/questions/51693427/sift-object-matching-in-python

No entanto, esses esforços para criar descritores genéricos foram, em grande parte, suplantados pela ascensão das CNNs. As CNNs são capazes de aprender automaticamente características úteis diretamente dos dados, eliminando a necessidade de um processo manual de extração de features. Essa capacidade de abstrair o processo de extração de features é uma das principais vantagens das CNNs em comparação com métodos tradicionais de visão computacional.

2 ARQUITETURAS CONVOLUCIONAIS

2.1 Estrutura básica

A revolução das CNNs teve seu marco inicial com a AlexNet, vencedora da competição ImageNet de 2012. A AlexNet demonstrou uma capacidade

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

surpreendente de reconhecimento de imagens em grande escala, impulsionada pela arquitetura profunda e pela aplicação eficiente de camadas convolucionais.

Uma CNN é composta por várias camadas, representada na Figura 5, sendo as principais as camadas **convolucionais** (retângulos azuis), de **pooling** (os cones) e **totalmente conectadas** (FC – *Fully Connected*).

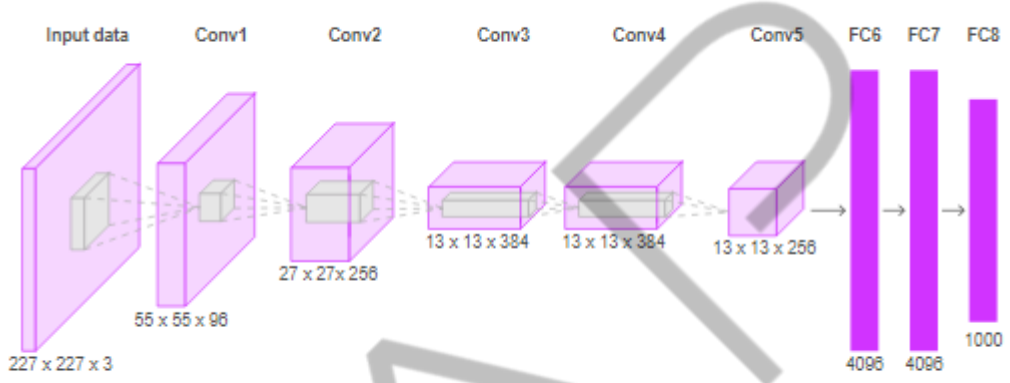


Figura 5– Representação da AlexNet
Fonte: Imagem reproduzida de: https://www.mdpi.com/2072-4292/9/8/848/htm

As **camadas convolucionais** são responsáveis por aprender padrões e características relevantes nas imagens, utilizando operações de convolução para aplicar filtros sobre a entrada. Em seguida, as **camadas de pooling** reduzem a dimensionalidade dos dados, preservando as características mais importantes. Por fim, as **camadas totalmente conectadas** processam essas características para realizar a classificação final.

Não há a necessidade de extrair features por meio de descritores, pois a extração ocorre automaticamente a partir dos pixels da imagem. O treinamento da rede visa garantir que a CNN está aprendendo a extrair features cada vez mais relevantes para a tarefa de destino (exemplo: classificar imagens como contendo gatos ou cachorros).

A flexibilidade e eficácia das CNNs revolucionaram o campo da inteligência artificial e continuam a ser uma área de intenso desenvolvimento e pesquisa.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Entendendo operações de Convolução e Pooling

Podemos afirmar, portanto, que as CNNs são redes profundas para extração de features. Mas como esse processo funciona? Essa extração está baseada nas operações de **convolução** e **pooling**.

Intuição: imagine que você tem um montão de LEGO e quer construir uma casa. Primeiro, você pega uma peça e vai encaixando outra em cima dela, uma por uma, até formar uma parede. Essa é a convolução, é como ir montando tijolinho por tijolinho. Depois, quando a parede está grande, você percebe que existem alguns tijolinhos pequenos e iguais juntinhos. Então, você substitui esses tijolinhos pequenos por um bloco maior. Isso é o pooling: juntar os tijolinhos parecidos para a casa ficar mais simples e rápida de montar.

Explicação matemática: convolução é uma operação matemática linear que, a partir de duas funções dadas, resulta numa terceira que mede **a soma do produto dessas funções** ao longo da região subentendida pela superposição delas *em função do deslocamento* existente entre elas.

Operações em uma Rede Neural: a operação de convolução em uma CNN é uma técnica que consiste em deslizar um filtro (também chamado de kernel) sobre a imagem de entrada, multiplicando os valores dos pixels da imagem pelos valores do filtro e somando os resultados para obter um único valor na saída, formando uma nova imagem (Figura 6).

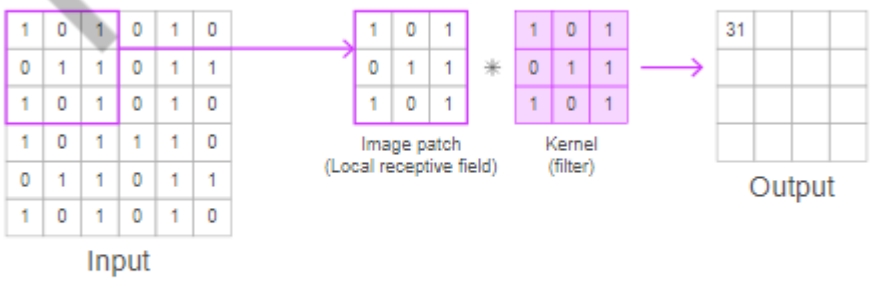


Figura 6 – Operação de convolução

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: Imagem adaptada de: https://www.superannotate.com/blog/guide-to-convolutional-neural-networks

Essa operação é repetida para diferentes regiões da imagem de entrada, produzindo uma imagem de saída que destaca padrões e características relevantes da imagem original (Figura 7).

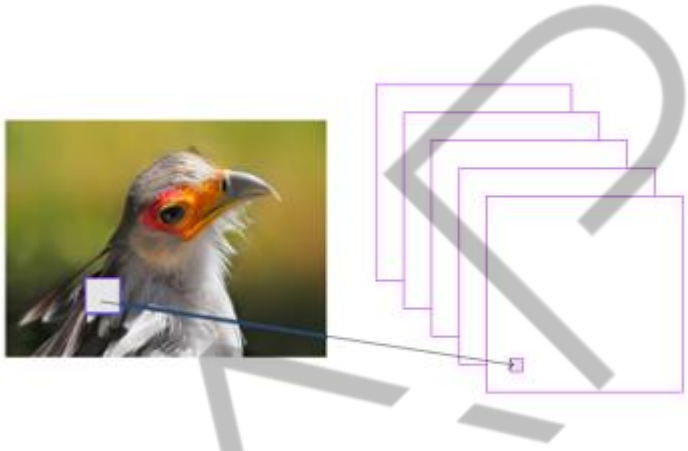


Figura 7 – Operação de convolução
Fonte: Imagem adaptada de: https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

Já o pooling é uma operação de subamostragem que reduz a dimensionalidade da imagem, preservando as características mais importantes. A operação de pooling, representada na Figura 8, divide a imagem em regiões e substitui cada região por um único valor, comumente o máximo ou a média dos valores da região. Isso ajuda a reduzir o custo computacional e o overfitting, além de fornecer invariância a pequenas variações na posição dos padrões na imagem.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

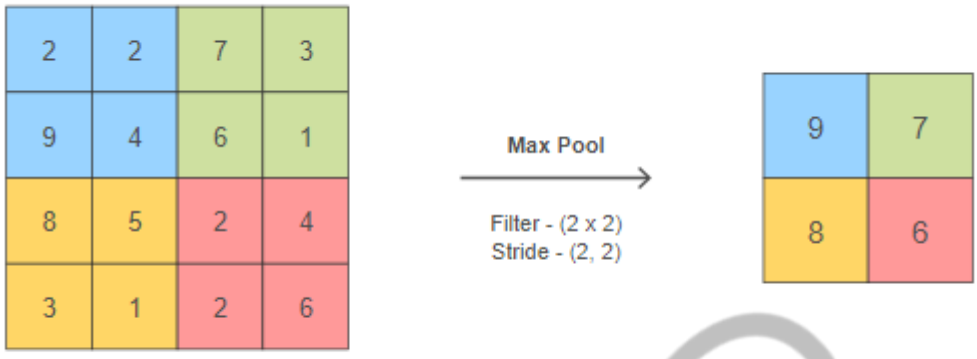


Figura 8 – Operação de pooling
Fonte: Imagem reproduzida de: https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/

A definição matemática de convolução indica um “deslocamento” entre as funções. Nas CNNs, esse deslocamento é dado pela janela deslizante do kernel sobre a imagem. Considere a Figura 9: ela ilustra a operação de convolução entre uma imagem 4 (linhas) x 5 (colunas) e um kernel 2x2. A cada passo, parte da imagem é considerada uma função, e filtro do kernel, outra função. Multiplicadas, elas produzem um escalar que está preenchendo outra matriz, a qual, após preenchida, será multiplicada pelo kernel novamente.

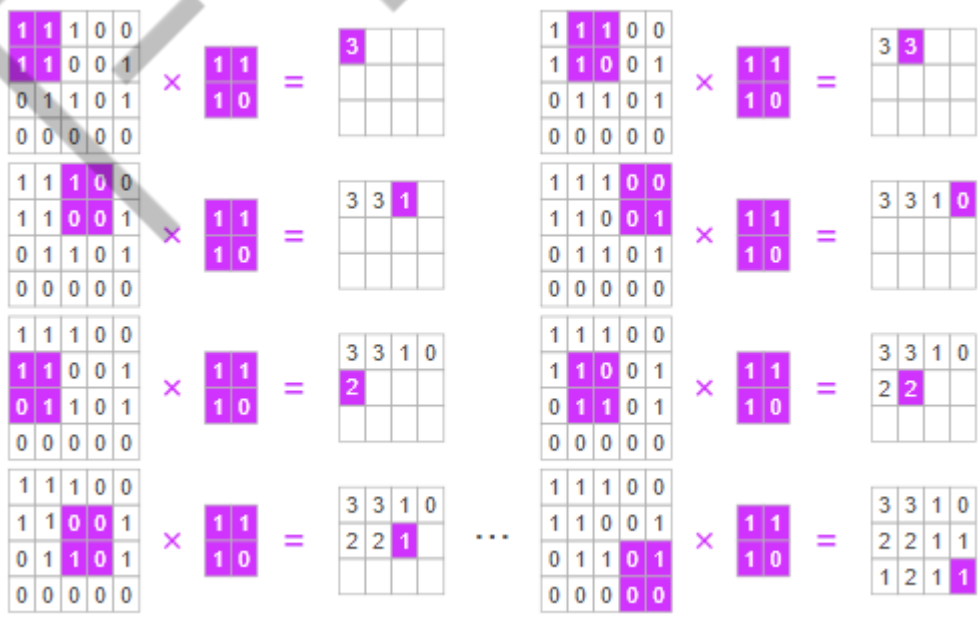


Figura 9 – Exemplo de convolução

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: Produzida pelo autor (2024)

Note que a convolução está ocorrendo da esquerda para a direita com um passo de 1 posição tanto para coluna quanto para linha. No final da imagem, temos a matriz resultante completamente preenchida.

Agora chegou a hora de fazer o pooling. A Figura 10 ilustra o processo iterativo. Uma vez estabelecido o tamanho da imagem resultante e das regiões a serem aplicadas o pooling, a operação mantém o valor mais representativo daquela região.

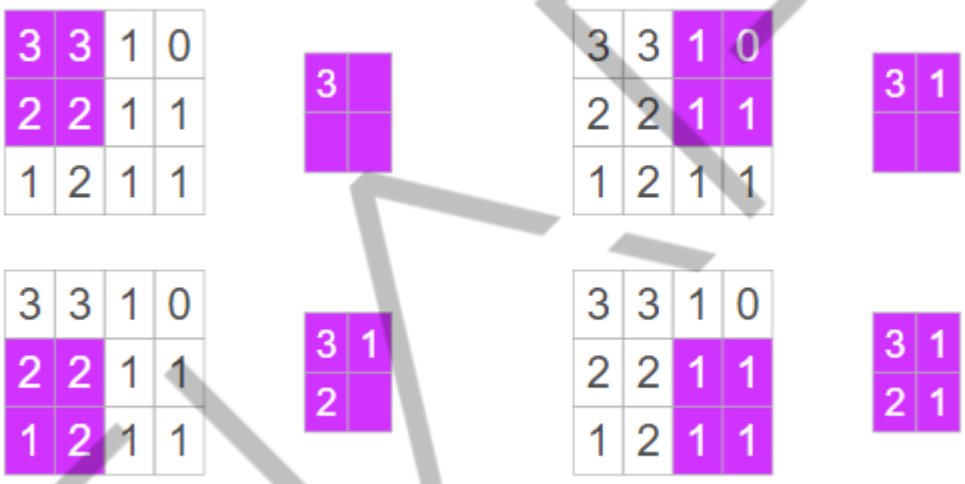


Figura 10 – Exemplo de pooling
Fonte: Produzida pelo autor (2024)

Existem formas distintas de aplicar o pooling, todas visam reduzir a dimensionalidade das features extraídas da imagem. Os mais comuns são:

- **Max Pooling**: no max pooling, uma região (geralmente 2x2) é percorrida pela imagem de entrada e o valor máximo dentro dessa região é mantido na saída. Essa técnica enfatiza características dominantes e invariantes de posição, como bordas e texturas distintas, mantendo-as na saída.
- **Average Pooling**: no average pooling, uma região (novamente, geralmente 2x2) é percorrida pela imagem de entrada e a média dos valores dentro dessa região é calculada e mantida na saída. Essa

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

técnica é mais suave e menos propensa a perder informações importantes, uma vez que considera todos os valores na região de pooling, em vez de apenas o máximo.

Mas quem disse que o melhor kernel 2x2 para essa imagem deveria ser aquele ilustrado em laranja na Figura 9? Pois é, não há garantias que esse kernel seja o mais indicado para todos os problemas de classificação de imagem. Portanto, ele é um parâmetro que precisa ser aprendido! De fato, ele é o peso da rede que precisa ser refinado durante o treinamento!

2.3 Configurações avançadas

Além do tamanho da convolução e a operação de pooling a ser aplicada, existem outros parâmetros a serem pensados na composição da arquitetura da CNN. Dentre eles, os mais relevantes são o **padding** (preenchimento) e o **striding** (passo), que afetam o tamanho e a resolução das saídas das camadas convolucionais. Por fim, a escolha das **funções de ativação** influencia como a máquina aprende.

2.3.1 Padding e Striding

O **padding** é uma técnica usada para controlar o tamanho da saída após a aplicação da operação de convolução. Ele consiste em adicionar zeros ao redor da borda da entrada antes de aplicar a convolução. O padding pode ser "*valid*" (sem padding) ou "*same*" (padding suficiente para manter a altura e largura na saída). A Figura 11 mostra o padding de 1, incluindo uma borda de 1 pixel ao redor da imagem.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

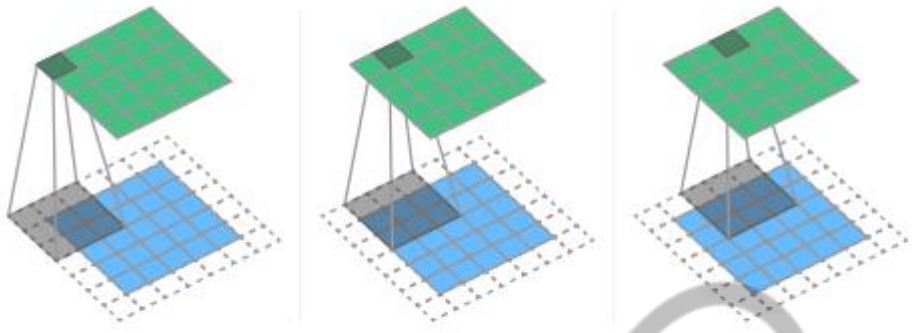


Figura 11 – Exemplo de padding
Fonte: Produzida pelo autor (2024)

O padding é importante para preservar informações nas bordas das imagens e evitar a diminuição do tamanho da saída após várias camadas de convolução.

O **striding** determina o quanto o filtro se move pela entrada a cada passo durante a operação de convolução. Um striding de 1 significa que o filtro se move pixel a pixel, enquanto um striding maior reduz a resolução da saída, pois o filtro salta mais pixels a cada passo. Um striding maior pode ajudar a reduzir o tempo de computação e o custo computacional, mas pode levar à perda de informações finas nas camadas convolucionais. A Figura 12 mostra um striding de tamanho 2. Note que o valor é aplicado tanto na largura quanto na altura da imagem.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

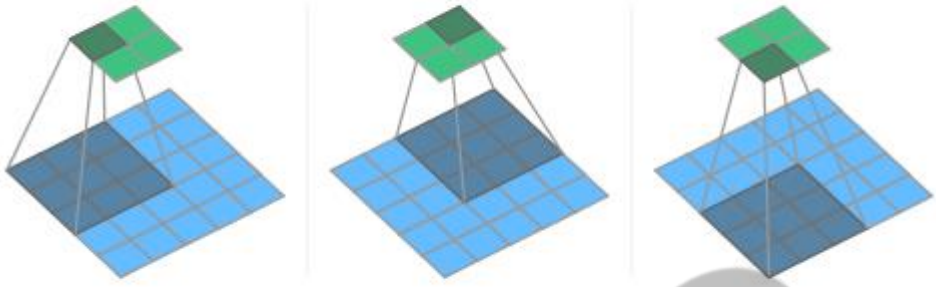


Figura 12 – Exemplo de striding
Fonte: Produzida pelo autor (2024)

A configuração adequada do padding e do striding pode ter impactos significativos no desempenho e na eficácia de uma CNN. O uso adequado de padding pode ajudar a preservar informações nas bordas das imagens e manter o tamanho da saída desejado. Por outro lado, um striding maior pode reduzir a resolução espacial da saída, o que pode ser útil para reduzir o custo computacional e a dimensionalidade dos dados, mas pode levar à perda de informações importantes.

2.3.2 Funções de ativação

As funções de ativação desempenham um papel crucial nas redes neurais por introduzirem não linearidades, permitindo que a rede aprenda e represente relações complexas nos dados. A Tabela 1 lista as funções de ativação mais comuns e o impacto de seu uso em CNNs.

Função de Ativação	Prós	Contras	Melhores práticas
ReLU (Rectified Linear Unit)	Simplicidade computacional, ajuda a mitigar o problema de vanishing gradients, acelera a convergência em	Pode levar a "dying ReLU" problem, onde os neurônios param de aprender devido a valores de entrada negativos constantemente	É amplamente utilizado em camadas convolucionais devido à sua eficácia e eficiência computacional.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	comparação com funções sigmoidais.	resultando em saídas zero.	
Leaky ReLU	Resolvem o problema do "dying ReLU", permitindo um pequeno gradiente para valores negativos.	Pode resultar em valores inconsistentes para gradientes negativos.	É útil em casos em que a ReLU padrão não converge bem.
ELU (Exponential Linear Unit)	Combate o problema do "dying ReLU", produzindo uma saída suave para valores negativos e permitindo gradientes suaves.	Mais computacionalmente caro devido ao uso de funções exponenciais.	Pode ser eficaz em redes mais profundas e em problemas que exigem suavidade nas ativações.
Sigmoid e Tangente Hiperbólica (tanh)	Produzem saídas no intervalo limitado (0 a 1 para sigmoid e 1 a 1 para tanh), útil para problemas de classificação binária.	Propensos ao problema de vanishing gradients, podem resultar em valores saturados nos gradientes.	Principalmente utilizados em camadas de saída para problemas de classificação binária.
Linear	Simplicidade, útil em camadas de saída para problemas de regressão.	Não introduz não linearidades, limitando a capacidade de aprendizado da rede.	Usado em camadas de saída para problemas de regressão, onde as previsões precisam ser contínuas.

Tabela 1 – Funções de ativação para CNNs
Fonte: Elaborado pelo autor (2024)

Em geral, a escolha da função de ativação depende do problema específico, da arquitetura da rede e da experimentação para determinar qual função funciona melhor para o caso em questão. Experimentação e ajuste fino são fundamentais para determinar a melhor função de ativação em uma CNN.

2.3 Terminologia e evolução

A terminologia das CNNs pode variar de acordo com diferentes características e dimensões. Os termos mais importantes são **profundidade** (*depth*), **largura** (*width*) e **resolução** (*resolution*). Esses conceitos estão ilustrados na Figura 13, sendo comparados com uma rede baseline.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

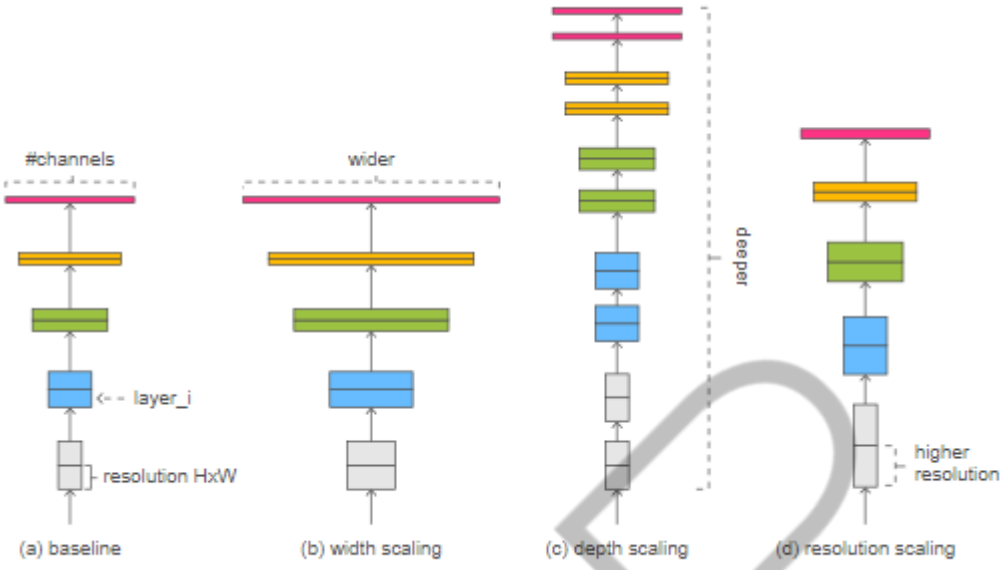


Figura 13 – Terminologias de CNNs
Fonte: Imagem reproduzida de M Tan, Q Le

Tais conceitos se referem a:

- **Redes Profundas**: redes com várias camadas, que podem incluir dezenas a centenas de camadas. Essas redes têm a capacidade de aprender representações hierárquicas de recursos nos dados, tornando-as ideais para tarefas complexas que exigem uma compreensão profunda das características intrínsecas dos dados;
- **Redes Largas**: redes com grande número de neurônios em cada camada. Essas redes têm uma grande capacidade de aprendizado, pois podem capturar uma ampla gama de características dos dados. No entanto, elas também podem ser mais propensas a overfitting, especialmente quando os conjuntos de dados são pequenos;
- **Redes de Maior Resolução**: redes que operam em imagens de alta resolução ou em outras formas de dados com alta dimensionalidade. Essas redes podem exigir camadas especiais de pré-processamento ou arquiteturas adaptadas para lidar com a complexidade dos dados de alta resolução.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A evolução das arquiteturas de CNNs tem sido marcada por avanços significativos ao longo dos anos, tanto em termos de profundidade quanto resolução e amplitude.

Inicialmente, as arquiteturas de CNNs eram relativamente rasas e simples, como a AlexNet, que introduziu o conceito de convoluções em profundidade e pooling. Com o tempo, arquiteturas mais avançadas foram desenvolvidas para lidar com desafios mais complexos e melhorar o desempenho em tarefas de visão computacional. Se traçarmos a precisão de todos os trabalhos relatados no ImageNet, obteríamos o gráfico da Figura 14:

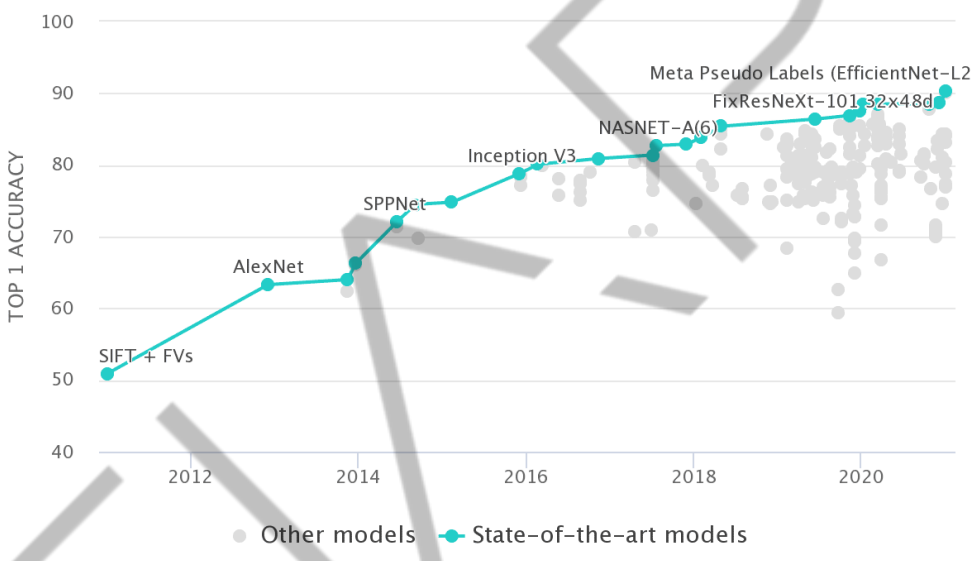


Figura 14 – Benchmark de CNNs
Fonte: Imagem reproduzida de: https://theaisummer.com/cnn-architectures/

O tamanho das redes cresceu e variou enormemente ao longo dos anos, com arquiteturas cada vez mais profundas e complexas. Por outro lado, redes mais simples foram propostas e conseguiram resultados extraordinários, ao passo de reduzirem o esforço computacional para treinamento. Para fornecer outra visão geral, seria possível capturar as CNNs de melhor desempenho até 2018 em uma única imagem, presente na Figura 15:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

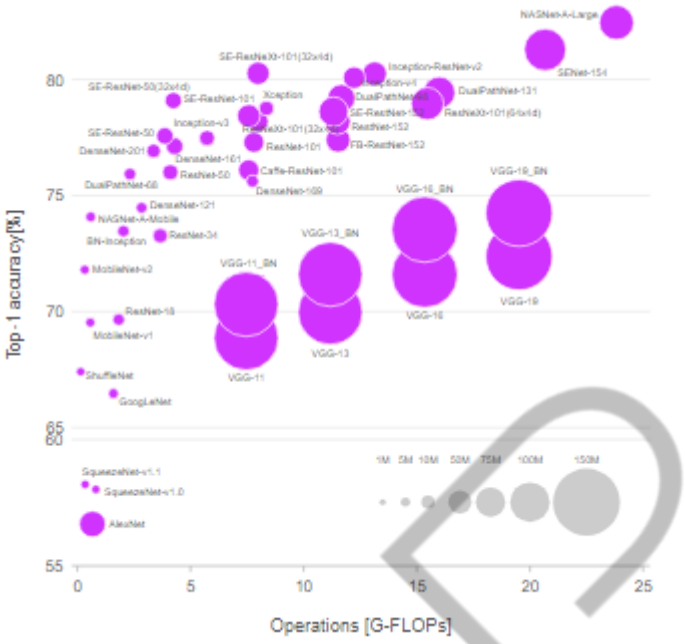


Figura 15 – Overview das arquiteturas CNNs até 2018
Fonte: Imagem reproduzida de: https://theaisummer.com/cnn-architectures/

Note, pela Figura 15, que a AlexNet – totalmente revolucionária em 2012 – está complemente obsoleta em 2018. Repare também que à medida que as redes foram ficando mais complexas, a performance aumentou. No entanto, mesmo redes não tão grandes conseguem desempenho “estado da arte” na competição (no mundo acadêmico, esta expressão significa que o resultado figura entre os melhores ou até mesmo o melhor). na competição. Portanto, profundidade não é sinônimo de desempenho, e a forma como as redes são conectadas e configuradas influenciam enormemente os resultados atingidos.

Algumas das arquiteturas mais modernas e influentes incluem:

- **VGGNet**: introduziu uma arquitetura profunda e uniforme com múltiplas camadas convolucionais, ajudando a estabelecer o padrão para redes mais profundas;
- **GoogLeNet (Inception)**: apresentou módulos de convolução inception, que permitem uma exploração eficiente de várias escalas de recursos e reduzem a dimensionalidade dos mapas de características intermediárias;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **ResNet**: introduziu blocos residuais, que permitem o treinamento de redes ainda mais profundas, mitigando o problema de vanishing gradients por meio de conexões de atalho;
- **EfficientNet**: utiliza escalonamento composto para otimizar o equilíbrio entre profundidade, largura e resolução da rede, alcançando melhor desempenho em comparação com arquiteturas convencionais.

Essas arquiteturas mais modernas incorporam técnicas inovadoras para melhorar a eficiência e o desempenho das CNNs, permitindo avanços significativos em uma ampla gama de aplicações de visão computacional, como segmentação e identificação de objetos.

3 TREINAMENTO DE REDES NEURAIS CONVOLUCIONAIS

Agora, exploraremos aspectos cruciais do treinamento das CNNs, essenciais para extrair o máximo desempenho desses modelos em uma variedade de tarefas de visão computacional.

A **preparação adequada dos dados** é fundamental para garantir que as CNNs sejam capazes de aprender padrões significativos nos dados de entrada, enquanto **técnicas de regularização** são essenciais para evitar overfitting e melhorar a generalização do modelo. Além disso, a **otimização de hiperparâmetros** desempenha um papel crucial na busca pela configuração ideal do modelo, maximizando sua capacidade de aprender e generalizar a partir dos dados disponíveis. Por fim, abordaremos também o treinamento de **redes 3D (convolução sobre volumes)**, que permite estender as capacidades das CNNs para dados volumétricos, abrindo novas possibilidades de aplicação em áreas como imagens médicas e processamento de vídeo.

Ao compreender e dominar esses aspectos do treinamento de CNNs, podemos aproveitar ao máximo o potencial do Deep Learning em tarefas diversas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3.1 Preparação de dados

A preparação de dados para CNNs envolve várias etapas, começando pela coleta e aquisição dos dados brutos. Em seguida, os dados geralmente passam por uma etapa de pré-processamento, onde são realizadas operações como **redimensionamento** para um tamanho padrão, **normalização** para garantir que todas as características tenham a mesma escala e, se necessário, **aumento de dados** para aumentar a variabilidade dos dados de treinamento.

Além disso, a **segmentação** pode ser aplicada para isolar regiões de interesse em imagens complexas. Outras técnicas de pré-processamento incluem a **remoção de ruído e artefatos indesejados**, bem como a correção de distorções ou irregularidades nos dados de entrada.

Uma consideração importante na preparação de dados para CNNs é a **organização adequada dos dados em conjuntos de treinamento, validação e teste**. É essencial que esses conjuntos sejam representativos da distribuição dos dados do mundo real e que sejam suficientemente grandes para permitir uma avaliação precisa do desempenho do modelo. Pode ser útil considerar a presença de classes desbalanceadas e garantir que o conjunto de dados inclua exemplos suficientes de cada classe.

3.2 Técnicas de regularização

No contexto das CNNs, a regularização é essencial para evitar que o modelo se torne muito complexo e super ajustado aos dados de treinamento, garantindo assim que ele generalize bem para novos dados e produza resultados mais confiáveis e robustos.

Uma técnica comum de regularização é a **Dropout**, que consiste em desligar aleatoriamente um certo número de unidades de neurônios durante o treinamento, reduzindo assim a coadaptação entre os neurônios e melhorando a generalização do modelo. Outras técnicas de regularização incluem a **Regularização L1**, que adiciona uma penalidade baseada na norma L1 dos pesos, e a **Regularização L2**, que adiciona

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

um termo à função de custo do modelo, penalizando os pesos grandes e incentivando pesos menores, o que reduz a complexidade do modelo.

Além disso, a **Normalização em Lotes (Batch Normalization)** é frequentemente utilizada como uma forma de regularização, normalizando as ativações de cada camada do modelo. Isso ajuda a estabilizar o treinamento e reduzir a dependência da inicialização dos pesos.

Finalmente, a **Redução da Taxa de Aprendizado** diminui gradualmente a taxa de aprendizado ao longo do treinamento para permitir uma busca mais precisa no espaço de parâmetros.

3.3 Otimização de hiperparâmetros

A otimização de hiperparâmetros é uma etapa crucial no desenvolvimento de modelos de CNNs, pois determina a configuração ideal do modelo, como a taxa de aprendizado, o tamanho do kernel da convolução, o número de camadas, padding, striding e tantos outros. A escolha adequada desses hiperparâmetros pode levar a um modelo mais eficiente e com melhor desempenho.

Uma abordagem comum para otimizar os hiperparâmetros é realizar uma busca em **grade** (grid search) ou **busca aleatória** (random search) sobre um espaço pré-definido de hiperparâmetros. Nesse processo, diferentes combinações de hiperparâmetros são avaliadas por meio de validação cruzada em um conjunto de dados de validação, e a combinação que leva ao melhor desempenho é selecionada.

Outra técnica é a **otimização bayesiana**, que utiliza métodos probabilísticos para encontrar a melhor combinação de hiperparâmetros de forma mais eficiente do que a busca em grade ou aleatória. Essa abordagem tende a convergir mais rapidamente para a região do espaço de hiperparâmetros que leva aos melhores resultados.

Além disso, técnicas como o **uso de regularização** e normalização em lotes também podem ser consideradas como parte do processo de otimização de hiperparâmetros, pois afetam diretamente o desempenho do modelo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Como em todo modelo de machine learning, esse é um processo iterativo muitas vezes computacionalmente intensivo, mas essencial para desenvolver modelos de alto desempenho e com boa capacidade de generalização. É importante explorar diferentes abordagens e técnicas para encontrar a configuração ideal.

3.4 Operações sobre volumes

Em CNNs de redes RGB, os volumes de dados representam imagens tridimensionais, onde cada canal (R, G, B) corresponde à uma dimensão. As operações sobre esses volumes incluem convoluções tridimensionais, onde um filtro é aplicado em todas as três dimensões do volume de entrada simultaneamente. Durante a convolução, o filtro é deslizado ao longo de todas as três dimensões, capturando padrões espaciais em todas as camadas de cores da imagem.

Além disso, as operações de pooling podem ser realizadas em cada canal individualmente ou em todo o volume. O pooling reduz a dimensionalidade do volume, preservando as características mais relevantes. Assim como nas CNNs tradicionais, as CNNs de redes RGB também podem usar padding e striding para controlar o tamanho e a taxa de redução do volume ao longo das operações de convolução e pooling. Essas operações são essenciais para extrair e preservar características importantes das imagens RGB durante o processo de aprendizado da rede.

Para calcular uma célula de saída, execute a convolução em cada canal correspondente e, em seguida, some o resultado.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

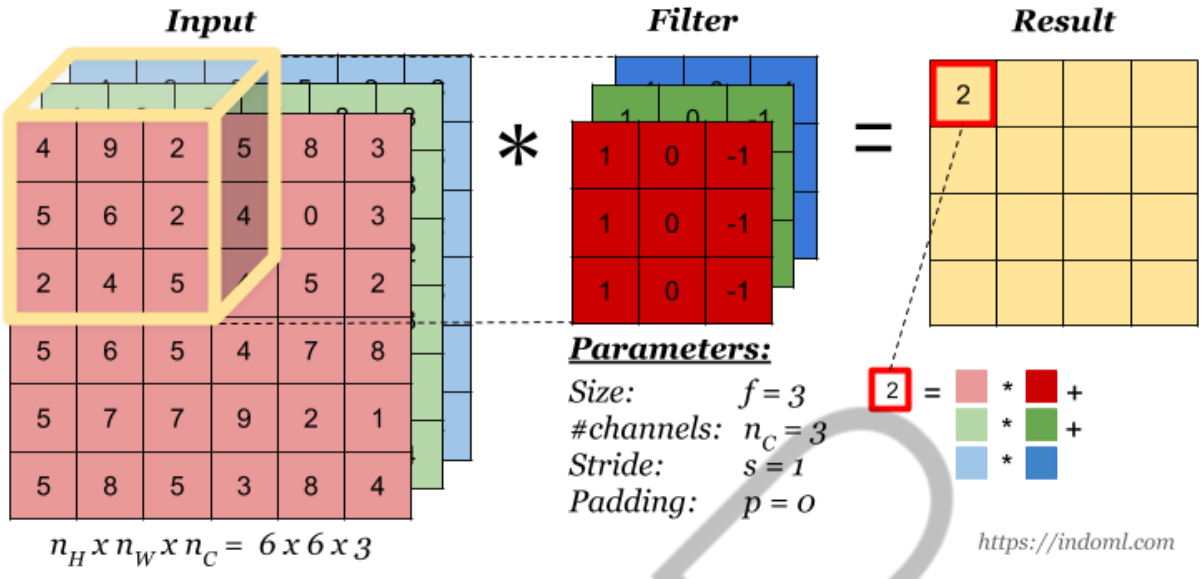


Figura 16 – Operação de Convolução em Volume
Fonte: Imagem reproduzida de: https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/

No caso da representação da Figura 16, o número total de multiplicações a serem calculadas para atingir o resultado é (4 x 4) x (3 x 3 x 3) = 432.

4 DESAFIOS E AVANÇOS EM REDES NEURAIS CONVOLUCIONAIS

Toda rede neural pode sofrer overfitting ou underfitting, e com as CNNs isso não é diferente. Ao longo do capítulo, apresentamos hiperparâmetros que auxiliam as redes convolucionais a extraírem cada vez mais informações dos dados (evitar underfitting), mas sempre ponderando sobre seu poder de generalização (evitar overfitting).

Ainda assim, as CNNs padecem de um grande problema: **a necessidade de grandes quantidades de dados para viabilizar seu treinamento**. Em alguns contextos, nem sempre é viável obter novos dados, seja pelos custos envolvidos ou pela “raridade” do fenômeno registrado na imagem.

Para contornar a baixa disponibilidade de dados e ainda assim obter bons resultados nos treinamentos das CNNs, existem duas estratégias comumente aplicadas: a **aumentação da base de dados** e a **transferência de aprendizado**.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4.1 Técnicas de ampliação da base de dados

A aumentação da base de dados (*data augmentation*) é uma técnica crucial para melhorar o desempenho e a generalização de modelos de Deep Learning, incluindo CNNs. Essa técnica consiste em criar amostras de treinamento a partir das amostras originais, aplicando transformações aleatórias nos dados de entrada. Isso aumenta a diversidade dos dados de treinamento e ajuda a reduzir o overfitting.

Existem várias técnicas de ampliação de dados comumente usadas em problemas de visão computacional, **como rotação, translação, espelhamento horizontal e vertical, zoom, alteração de brilho e contraste, corte aleatório e adição de ruído**. Essas transformações são aplicadas de forma aleatória e controlada durante o treinamento do modelo, permitindo que ele aprenda a reconhecer objetos em diferentes posições, orientações e condições de iluminação.

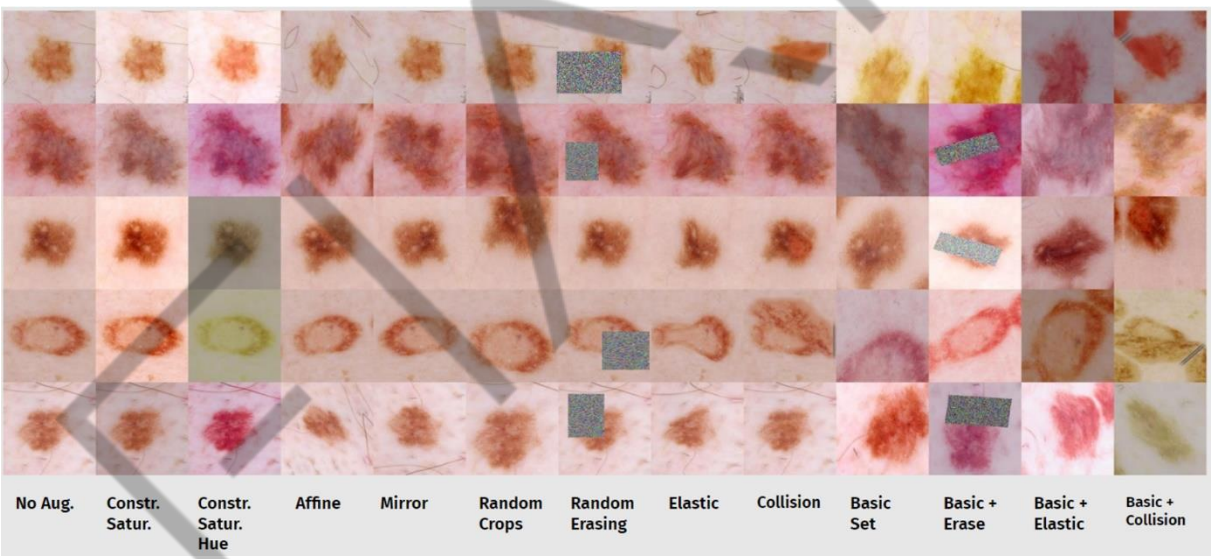


Figura 17 – Exemplos de aumentação de dados
Fonte: Fábio Perez

Veja o resultado da aplicação de tais técnicas de aumentação na Figura 17. Cada linha se refere à mesma imagem de lesão de pele, sendo que em cada coluna está o resultado de uma técnica de aumentação, eventualmente combinadas. A primeira coluna representa a imagem original, sem aumentação alguma.

Dando um passo além, no sentido de usar técnicas mais sofisticadas, é possível **treinar GANs (*Generative Adversarial Network* – Redes Generativas**

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Adversárias) para gerar amostras sintéticas que são indistinguíveis das amostras reais.

Isso permite criar um número ilimitado de exemplos de treinamento adicionais, aumentando significativamente a diversidade e a quantidade de dados disponíveis para treinar modelos de Deep Learning, conforme ilustrado na Figura 18. As quatro lesões desta figura são falsas, e foram geradas por meio de uma GAN.

Figura 18 – Exemplos imagens geradas com GANs
Fonte: Imagem reproduzida de: https://arxiv.org/pdf/1902.03253.pdf

Outra vantagem do uso das GANs é sua capacidade de aprender a distribuição subjacente dos dados de treinamento e gerar amostras que capturam as características essenciais dos dados reais. Isso ajuda a evitar o overfitting e melhora a capacidade de generalização do modelo.

No entanto, é importante observar que treinar GANs pode ser desafiador e requer cuidadosa sintonia de hiperparâmetros, além de grandes conjuntos de dados de treinamento. Além disso, é essencial garantir que as amostras geradas sejam de alta qualidade e representem adequadamente a distribuição dos dados reais.

4.2 Transfer Learning e Fine-tuning

Transfer Learning e Fine-tuning são técnicas essenciais em Deep Learning, especialmente em CNNs, que permitem aproveitar o conhecimento adquirido em uma tarefa para melhorar o desempenho em outra tarefa relacionada.

As camadas convolucionais e de pooling de uma CNN funcionam basicamente como um extrator de features, as quais podem ser aprendidas por qualquer método de machine learning, incluindo redes neurais totalmente conectadas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O **transfer Learning** envolve o uso de um modelo pré-treinado em uma grande base de dados, geralmente em uma tarefa semelhante àquela que estamos enfrentando. O modelo pré-treinado é então ajustado (ou "transferido") para a nova tarefa, mantendo os pesos aprendidos nas camadas mais profundas e ajustando apenas as camadas finais para se adaptarem aos novos dados.

Figura 19 – Conceito de Transfer Learning
Fonte: Imagem reproduzida de: https://arxiv.org/pdf/1604.04024.pdf

Na Figura 19, imagine que uma CNN foi treinada do zero em uma base de dados muito grande (ex: ImageNet) para aprender a reconhecer objetos quaisquer. A mesma arquitetura foi usada para classificar imagens de lesões de pele entre "melanoma" ou "não melanoma". Os pesos treinados da primeira rede foram usados para extrair características do segundo cenário, e apenas as últimas camadas (de classificação) foram retreinadas para ajuste à tarefa de destino.

Por outro lado, o **Fine-tuning** é uma técnica que envolve o ajuste fino de um modelo pré-treinado em uma tarefa específica. Isso geralmente envolve o descongelamento de algumas camadas do modelo pré-treinado e o retreinamento do modelo em novos dados, com uma taxa de aprendizado menor para evitar grandes alterações nos pesos já aprendidos. Em outras palavras, é como se a rede fosse retreinada na tarefa de destino, porém não partindo de pesos aleatórios, mas de pesos que comprovadamente produzem boas features.

A importância dessas técnicas reside na capacidade de **reduzir o tempo e os recursos necessários para treinar modelos a partir do zero**, além de **melhorar o desempenho do modelo** em tarefas com conjuntos de dados menores. Elas são

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

altamente recomendadas quando temos um conjunto de dados pequeno ou limitado, pois permitem que o modelo se beneficie do conhecimento prévio adquirido em tarefas semelhantes.

5 MÃO NA MASSA! DEEP LEARNING EM PYTHON

Agora que conhecemos as CNNs, está na hora de praticarmos sua implementação. Nesta atividade, começaremos a criar redes neurais profundas, via TensorFlow. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: Veja o material HOW TO para acessar o ambiente do Colab.

DICA: Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 7) e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

DICA: Para essa parte do código, utilizaremos um dataset já salvo na memória do TensorFlow, por isso não há arquivos externos associados!

Utilizaremos a base “tf_flowers” já persistida no TensorFlow. Essa base contém imagens de cinco classes diferentes de flores: rosas, margaridas, dente de leão, girassóis e tulipas. Cada classe contém um número variável de imagens de alta resolução, capturadas de diferentes ângulos e condições de iluminação. Essa base de dados é frequentemente usada para tarefas de classificação de imagens, como treinamento e avaliação de modelos de CNNs para reconhecimento de flores. Exemplos de dados dessa base estão na Figura 20:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

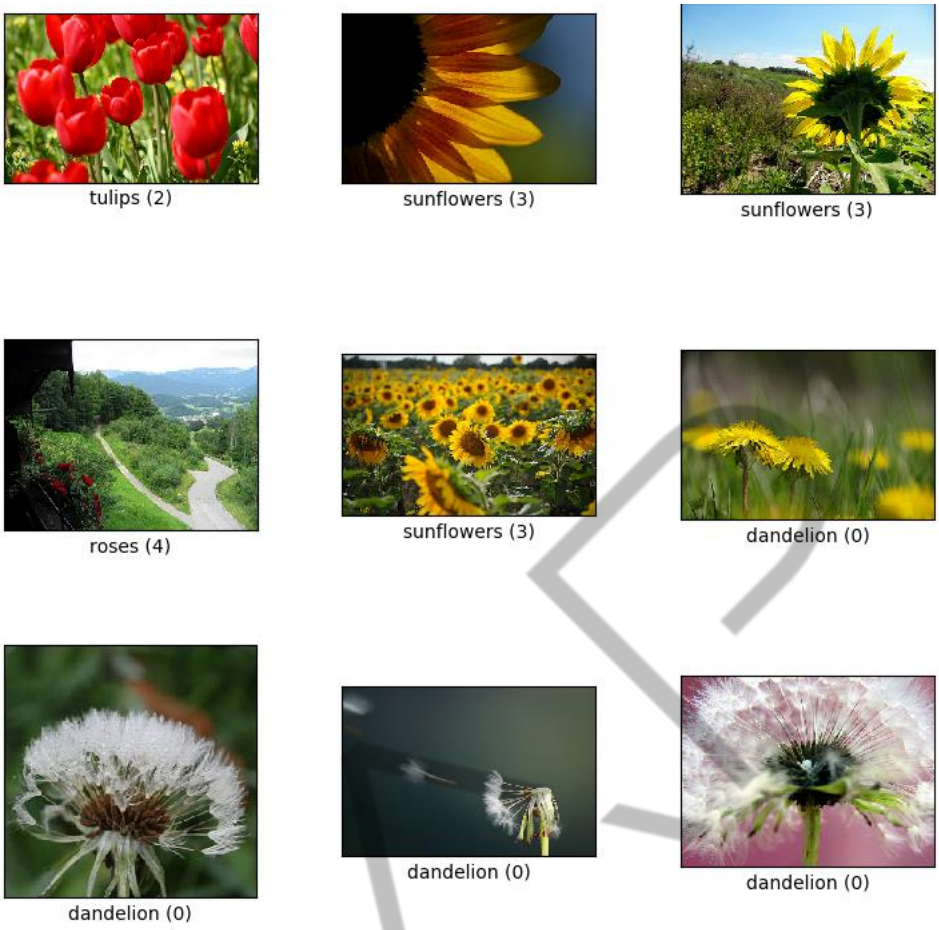


Figura 20– Exemplos da base TF_Flowers
Fonte: Imagem adaptada de: https://www.tensorflow.org/datasets/catalog/tf_flowers?hl=pt-br

Vamos começar com os *imports*:

```
# Imports
import tensorflow as tf
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix

from keras.api._v2.keras import callbacks
from tensorflow.keras import Sequential, layers
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers.experimental.preprocessing import
Rescaling

from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input

from tensorflow.keras import layers, models
```

Código-fonte 1 – Imports do código de CNN
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Esse primeiro código não gera saída. Vamos carregar o dataset:

```
# Carregar o dataset
import tensorflow_datasets as tfds
from tensorflow.keras.utils import to_categorical

(train_ds, train_labels), (test_ds, test_labels) = tfds.load(
    "tf_flowers",
    split=["train[:70%]", "train[:30%]"],
    batch_size=-1,
    as_supervised=True
)
```

Código-fonte 2– Carregando o dataset
Fonte: Elaborado pelo autor (2024)

O carregamento pode demorar alguns segundos e quando concluído, mostrará a barra de status em 100%. Note que já estamos definindo as variáveis que armazenarão o TREINO e o TESTE, bem como os LABELS. Caso você queira exibir o tamanho desses conjuntos, execute:

```
# Verificando o tamanho das variáveis
display("Tamanho do treino: ")
display(train_ds.shape)
display("Tamanho do treino: ")
display(test_ds.shape)
display("Labels do treino: ")
display(train_labels)
```

Código-fonte 3 – Chegando o tamanho dos conjuntos de treino e teste
Fonte: Elaborado pelo autor (2024)

O qual mostra que o TREINO possui 2569 imagens, enquanto o TESTE possui 1101. No entanto, em ambos os conjuntos, as imagens possuem 442x1024 pixels e são expressas em 3 canais (RGB). O tamanho do vetor de labels do treino também possui 2569 rótulos, que são números de 0 a 4 (representando as 5 classes do problema).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
'Tamanho do treino: '
TensorShape([2569, 442, 1024, 3])
'Tamanho do treino: '
TensorShape([1101, 442, 1024, 3])
'Labels do treino: '
<tf.Tensor: shape=(2569,), dtype=int64, numpy=array([2, 3, 3, ..., 0, 2, 0])>
```

Figura 21– Saída do Código-fonte 3
Fonte: Elaborado pelo autor (2024)

Agora vamos reescalar as imagens para um tamanho menor, fazendo com que a rede treine mais rápido. Imagens de alta resoluções demandam maior poder computacional. Queremos que as imagens fiquem com 150x150 pixels, mantendo o aspecto das cores. Também representaremos os labels de forma categórica. Assim, o label "`0`" vira o vetor "`[1., 0., 0., 0., 0.]`", o label "`1`" vira o vetor "`[0., 1., 0., 0., 0.]`", e assim sucessivamente:

```
# Tamanho alvo: 150x150 pixels
size = (150, 150)

# Redimensionando os conjuntos de treino e teste
train_ds = tf.image.resize(train_ds, size)
test_ds = tf.image.resize(test_ds, size)

# Aplicando a representação categórica dos labels
train_labels = to_categorical(train_labels, num_classes=5)
test_labels = to_categorical(test_labels, num_classes=5)

# Verificando o tamanho das variáveis
display("Novas dimensões do treino: ")
display(train_ds.shape)
display("Nova representação dos labels: ")
display(train_labels)
```

Código-fonte 4 – Pré-processamento dos dados
Fonte: Elaborado pelo autor (2024)

A saída será a exibida na Figura 22, a qual confirma que as imagens foram reescaladas e os labels foram recodificados, conforme solicitado:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
'Novas dimensões do treino: '
TensorShape([2569, 150, 150, 3])
'Nova representação dos labels: '
array([[0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0.],
       ...,
       [1., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [1., 0., 0., 0., 0.]], dtype=float32)
```

Figura 22 – Saída do Código-fonte 4
Fonte: Elaborado pelo autor (2024)

Vamos criar o primeiro modelo “profundo”. Ele não terá tantas camadas quanto os modelos modernos, mas exemplificará como encadear camadas convolucionais e de pooling. A rede criada à mão será sequencial, tendo uma camada após a outra. Ela começa com o reescalamento dos pixels, dividindo-os por 255. Aqui, já informamos que receberemos imagens 150x150x3.

Feito isso, adicionamos camada a camada, sempre combinando uma camada convolucional com um max pooling. Aqui, não há regras: podemos encadear várias convoluções, fazer poolings seguidos... estamos definhando uma arquitetura! A rede termina com camadas totalmente conectadas para fazer a classificação.

```
# Definir a arquitetura da CNN -> camada de extração de features
hand_model = Sequential()
hand_model.add( Rescaling(1./255, input_shape=(150, 150, 3)) )

hand_model.add( layers.Conv2D(16, kernel_size=10, activation="relu") )
hand_model.add( layers.MaxPooling2D(3) )

hand_model.add( layers.Conv2D(32, kernel_size=8, activation="relu") )
hand_model.add( layers.MaxPooling2D(2) )

hand_model.add( layers.Conv2D(32, kernel_size=6, activation="relu") )
hand_model.add( layers.MaxPooling2D(2) )

# Definir a arquitetura da CNN -> camada de decisão (classificação)
hand_model.add( layers.Flatten() )                          # entrada
hand_model.add( layers.Dense(50, activation="relu") )     # 1ª camada
oculta
hand_model.add( layers.Dense(20, activation="relu") )     # 2ª camada
oculta
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
hand_model.add( layers.Dense(5, activation="softmax") )  # camada de
decisão

# Compilar o modelo
hand_model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Definir a estratégia de Early Stopping
es = EarlyStopping(
    monitor="val_accuracy",
    patience=5,
    restore_best_weights=True
)

# Treina o modelo
with tf.device("/device:GPU:0"):
  hand_model.fit(
      train_ds, train_labels,
      epochs=50,
      validation_split=0.2,
      batch_size=32,
      callbacks=[es]
  )
```

Código-fonte 5 – Treinamento do modelo baseline, criado à mão
Fonte: Elaborado pelo autor (2024)

Uma vez que a arquitetura está montada, temos que compilar o modelo e treiná-lo (usando GPU!). Para agilizar, definiremos um Early Stopping se o treinamento não melhorar significamente após 5 épocas.

A saída será a exibida na Figura 23, indicando que o treinamento parou após 18 épocas:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
Epoch 11/50
65/65 [==============================] - 1s 19ms/step - loss: 0.8763 - accuracy: 0.6433 - val_loss: 1.1988 - val_accuracy: 0.5350
Epoch 12/50
65/65 [==============================] - 1s 19ms/step - loss: 0.8669 - accuracy: 0.6574 - val_loss: 1.0833 - val_accuracy: 0.5720
Epoch 13/50
65/65 [==============================] - 1s 19ms/step - loss: 0.7719 - accuracy: 0.7041 - val_loss: 1.1179 - val_accuracy: 0.6051
Epoch 14/50
65/65 [==============================] - 1s 19ms/step - loss: 0.7255 - accuracy: 0.7178 - val_loss: 1.2479 - val_accuracy: 0.5875
Epoch 15/50
65/65 [==============================] - 1s 21ms/step - loss: 0.6484 - accuracy: 0.7431 - val_loss: 1.1360 - val_accuracy: 0.5700
Epoch 16/50
65/65 [==============================] - 1s 21ms/step - loss: 0.5974 - accuracy: 0.7655 - val_loss: 1.1533 - val_accuracy: 0.5720
Epoch 17/50
65/65 [==============================] - 1s 19ms/step - loss: 0.5474 - accuracy: 0.7922 - val_loss: 1.3480 - val_accuracy: 0.5875
Epoch 18/50
65/65 [==============================] - 1s 19ms/step - loss: 0.4855 - accuracy: 0.8185 - val_loss: 1.5315 - val_accuracy: 0.5350
```

Figura 23 – Saída do Código-fonte 5
Fonte: Elaborado pelo autor (2024)

Vamos para a avaliação! Como é uma tarefa de classificação, calcularemos a acurácia e a matriz de confusão:

```
# Avaliação do modelo
loss, acc = hand_model.evaluate(test_ds, test_labels)

print(f"A loss do modelo é {loss:.2f} e a ACC é {acc:.2f}")
```

Código-fonte 6 – Calculando a acurácia do modelo inicial
Fonte: Elaborado pelo autor (2024)

A acurácia do modelo feito à mão é:

```
35/35 [==============================] - 1s 15ms/step - loss: 0.6375 - accuracy: 0.7629
A loss do modelo é 0.64 e a ACC é 0.76
```

Figura 24 – Saída do Código-fonte 6
Fonte: Elaborado pelo autor (2024)

Em termos de matriz de confusão, precisaremos do seguinte código:

```
# Função para obter a matriz de confusão
def plot_confusion_matrix(y_test, y_pred):
  labels = list( map( np.argmax, y_test ) )
  labels_pred = list( map( np.argmax, y_pred ) )

  cf_matrix = confusion_matrix(labels, labels_pred)
  sns.heatmap(cf_matrix, annot=True)


# Matriz de confusão
preds = hand_model.predict(test_ds)
plot_confusion_matrix(test_labels, preds)
```

Código-fonte 7– Plotando a matriz de confusão do modelo inicial
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Que gera a saída da Figura 25:

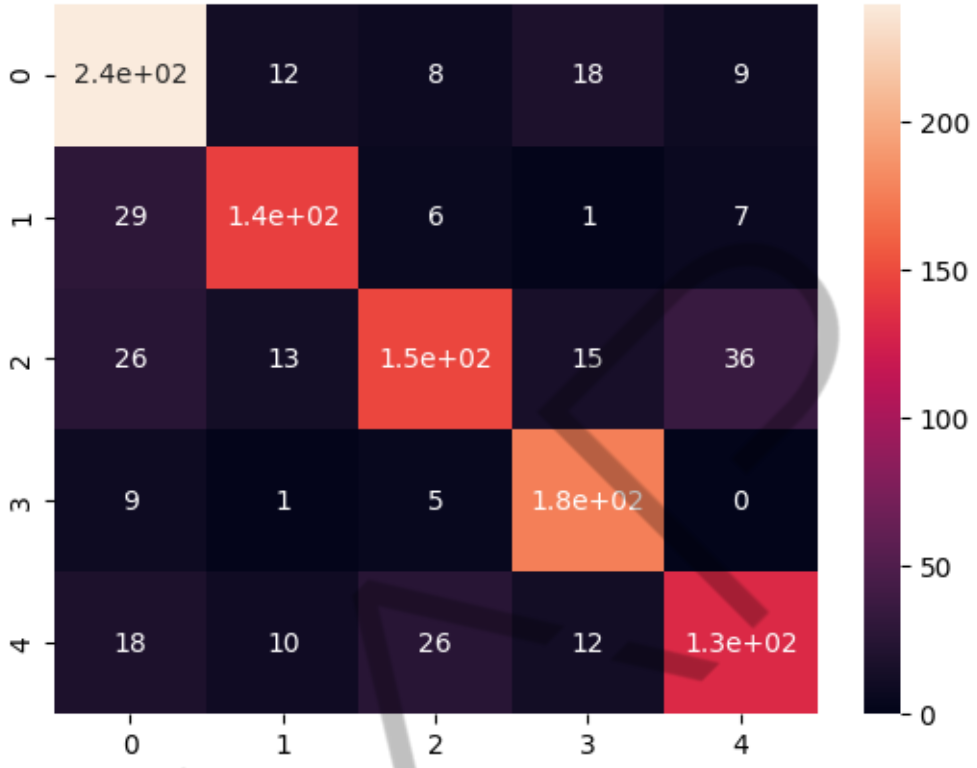


Figura 25 – Saída do Código-fonte 7
Fonte: Elaborado pelo autor (2024)

Para um modelo simples, é um bom resultado! Mas podemos fazer melhor! Vamos usar a técnica de Transfer Learning, usando um modelo de base poderoso para extrair boas features! Nosso transfer será realizado a partir do modelo VGG16.

Primeiramente, temos que pré-processar as imagens trazendo-as para a mesma distribuição dos dados que treinaram essa VGG, que no caso foi o ImageNet. Note, no nosso import, que o método “preprocess_input()” foi importado da VGG16 do TensorFlow. Esse ajuste é extremamente importante sempre que formos realizar o transfer de outro modelo.

```
# Preprocessa os dados de treinamento e teste
train_dsTL = preprocess_input(train_ds)
test_dsTL = preprocess_input(test_ds)

# Importar o modelo de base, que será a VGG16 ***sem*** a camada de
decisão
base_model = VGG16(weights="imagenet", include_top=False,
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                    input_shape=train_dsTL[0].shape)

# Usar os mesmos pesos da rede treinada (sem fazer fine tuning)
base_model.trainable = False
```

Código-fonte 8 – Pré-processamento dos dados para uso da VGG16 para transfer learning
Fonte: Elaborado pelo autor (2024)

O código também importa a VGG16 como modelo de base, usando os pesos treinados no ImageNet, mas sem incluir a camada “top”, que são as camadas de classificação. Ao indicarmos que o modelo não é “treinável”, estamos informando ao TensorFlow para usar os mesmos pesos da rede já pré-treinada, ou seja, use todas as camadas do modelo de base como um grande extrator de features.

A saída dessa célula é apenas o “download” dos pesos pré-configurados. O mais interessante é observarmos a composição do modelo de base:

```
# Observa a estrutura do modelo importado
base_model.summary()
```

Código-fonte 9 – Observando as camadas da VGG16
Fonte: Elaborado pelo autor (2024)

A saída está na Figura 26. Note quantas camadas existem no modelo VGG16.

```
Model: "vgg16"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 150, 150, 3)]     0

 block1_conv1 (Conv2D)       (None, 150, 150, 64)      1792

 block1_conv2 (Conv2D)       (None, 150, 150, 64)      36928

 block1_pool (MaxPooling2D)  (None, 75, 75, 64)        0

 block2_conv1 (Conv2D)       (None, 75, 75, 128)       73856

 block2_conv2 (Conv2D)       (None, 75, 75, 128)       147584

 block2_pool (MaxPooling2D)  (None, 37, 37, 128)       0

 block3_conv1 (Conv2D)       (None, 37, 37, 256)       295168

 block3_conv2 (Conv2D)       (None, 37, 37, 256)       590080

 block3_conv3 (Conv2D)       (None, 37, 37, 256)       590080

 block3_pool (MaxPooling2D)  (None, 18, 18, 256)       0
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
 block4_conv1 (Conv2D)        (None, 18, 18, 512)       1180160

 block4_conv2 (Conv2D)        (None, 18, 18, 512)       2359808

 block4_conv3 (Conv2D)        (None, 18, 18, 512)       2359808

 block4_pool (MaxPooling2D)   (None, 9, 9, 512)         0

 block5_conv1 (Conv2D)        (None, 9, 9, 512)         2359808

 block5_conv2 (Conv2D)        (None, 9, 9, 512)         2359808

 block5_conv3 (Conv2D)        (None, 9, 9, 512)         2359808

 block5_pool (MaxPooling2D)   (None, 4, 4, 512)         0

=================================================================
Total params: 14714688 (56.13 MB)
Trainable params: 0 (0.00 Byte)
Non-trainable params: 14714688 (56.13 MB)
```

Figura 26 – Saída do Código-fonte 9
Fonte: Elaborado pelo autor (2024)

Note também que esse modelo possui mais de14 milhões de parâmetros! No entanto, nenhum deles é “treinável”, pois já estão todos treinados. Usaremos esses mais de 14 milhões de pesos já ajustados no ImageNet como extratores de features das imagens de flores! O que falta, então, é definirmos a camada de decisão (classificação):

```
# Define as camadas
flatten_layer = layers.Flatten()
dense_layer_1 = layers.Dense(50, activation="relu")
dense_layer_2 = layers.Dense(20, activation="relu")
prediction_layer = layers.Dense(5, activation="softmax")

# Estabelece o modelo completo
model = models.Sequential( [
    base_model,         # A base CNN vem da VGG16
    flatten_layer,      # Achata as features que saem da VGG16
(base_model)
    dense_layer_1,      # 1ª camada oculta fully connected
    dense_layer_2,      # 2ª camada oculta fully connected
    prediction_layer    # Camada de saída, com 5 neurônios para
classificação
] )

model.summary()
```

Código-fonte 10 – Completando o modelo de transfer com camadas de classificação
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A saída está na Figura 27. Note que o modelo completo possui o início da VGG e depois as camadas de decisão que adicionamos, exatamente iguais ao modelo definido à mão (baseline).

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 vgg16 (Functional)          (None, 4, 4, 512)         14714688

 flatten_1 (Flatten)         (None, 8192)              0

 dense_3 (Dense)             (None, 50)                409650

 dense_4 (Dense)             (None, 20)                1020

 dense_5 (Dense)             (None, 5)                 105

=================================================================
Total params: 15125463 (57.70 MB)
Trainable params: 410775 (1.57 MB)
Non-trainable params: 14714688 (56.13 MB)
```

Figura 27– Saída do Código-fonte 10
Fonte: Elaborado pelo autor (2024)

Nessa saída, fica mais evidente que a quantidade de parâmetros aumentou, sendo que continuamos com mais de 14 milhões de parâmetros já treinados, porém temos mais de 410 mil a serem aprendidos, referentes às camadas de decisão.

A partir desse ponto, o processo de compilar e treinar o modelo é semelhante. Para uma comparação justa, também acionaremos o EarlyStopping após 5 épocas sem alterações profundas na métrica de validação.

```
# Compilar o modelo
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Definir a estratégia de Early Stopping
es = EarlyStopping(
    monitor="val_accuracy",
    patience=5,
    restore_best_weights=True
)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Treina o modelo
with tf.device("/device:GPU:0"):
  model.fit(
      train_dsTL, train_labels,  # alterei o nome do conjunto de
treino, usando o train_dsTL
      epochs=50,
      validation_split=0.2,
      batch_size=32,
      callbacks=[es]
  )
```

Código-fonte 11– Compilando o treinando o modelo resultante
Fonte: Elaborado pelo autor (2024)

A saída é o histórico de métricas em tempo de treinamento. Vamos direto para a avaliação, também em termos de acurácia e matriz de confusão:

```
# Avaliação do modelo: acurácia
loss, acc = model.evaluate(test_dsTL, test_labels)

print(f"A loss do modelo é {loss:.2f} e a ACC é {acc:.2f}")

# Avaliação do modelo: matriz de confusão
preds = model.predict(test_dsTL)
plot_confusion_matrix(test_labels, preds)
```

Código-fonte 12– Avaliação do modelo com transfer learning
Fonte: Elaborado pelo autor (2024)

A saída esperada está na Figura 28:

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

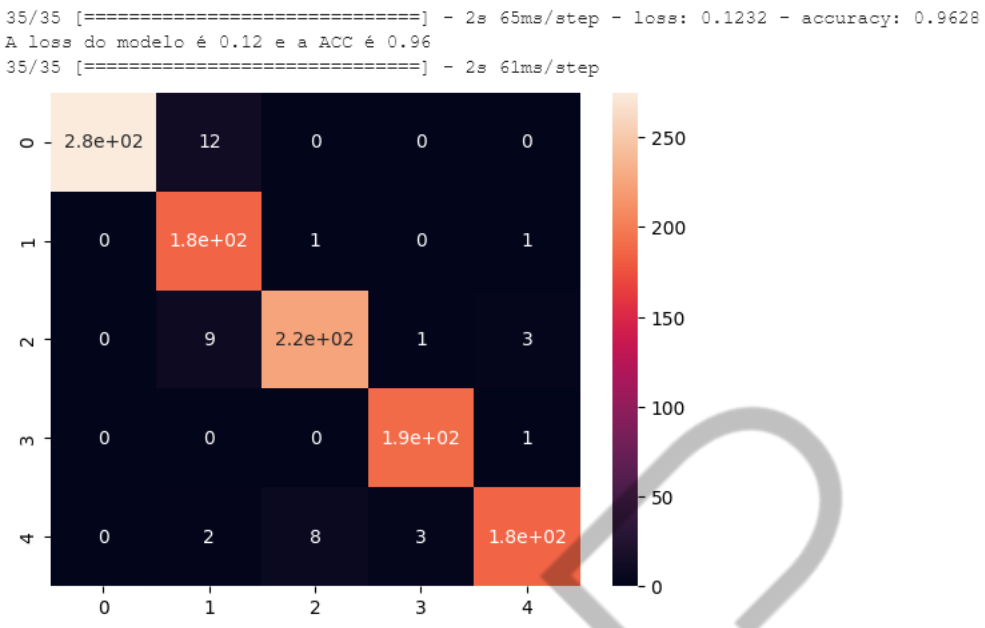


Figura 28 – Saída do Código-fonte 12
Fonte: Elaborado pelo autor (2024)

Muito melhor, não é mesmo? Nossa acurácia subiu para 96% e quase não há confusão entre as classes!

Esse código nos mostrou que o transfer learning é um artifício valioso para gerar modelos profundos sem precisar treinar do zero as redes, mas aproveitando modelos pré-treinados como bons extratores genéricos de features, os quais podem ser “aprendidos” por outros algoritmos de machine learning, incluindo redes neurais totalmente conectadas.

6 HANDS ON: PRATICANDO O USO DE CNNS

6.1 Aquecimento

Neste capítulo, aprendemos como construir modelos preditivos baseados em CNNs. Antes de começar o projeto, aproveite para estudar o **tutorial do próprio TensorFlow sobre Transfer Learning e Fine-tuning**. Esse tutorial está muito bem documentado e será outra oportunidade para você aprender como fazer Transfer Learning de outras arquiteturas previamente treinadas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Além disso, o tutorial ensina **como realizar o Fine-tuning**, descongelando algumas camadas e permitindo que os pesos sejam reacomodados para a tarefa de destino.

Acesse o tutorial abaixo e acompanhe com atenção. Não deixe de vinculá-lo ao Google Colab e executar o passo a passo com GPU, compreendendo o que está acontecendo.

Link: https://www.tensorflow.org/tutorials/images/transfer_learning?hl=pt-br

Esse notebook aplica CNN para predizer outra tarefa de classificação, mas agora fazendo o Transfer Learning a partir da MobileNet. O tutorial também compara os resultados do transfer puro com o fine-tuning, mostrando que o “ajuste fino” faz diferença nas medidas de desempenho.

DICA: Para alterar seu ambiente pra contendo uma GPU no Google Colab, basta clicar no ambiente de execução, depois em “Alterar o tipo de ambiente de execução” e então selecionar “T4 GPU” como acelerador de hardware.

6.2 Seu desafio

Sua tarefa é criar um modelo de classificação de câncer de mama por meio da análise de imagens histopatológicas. O conjunto de dados a ser utilizado, "*Breast Histopathology Images*", está disponível no Kaggle. Ele consiste em imagens digitais de tecido mamário humano. As imagens estão em formato de alta resolução e são usadas para análise e diagnóstico de câncer de mama por patologistas. Cada imagem representa uma amostra de tecido mamário e é rotulada como "negativa" se não contiver células cancerosas ou "positiva" se contiver células cancerosas.

A descrição completa do desafio está no notebook disponibilizado como material de apoio.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

DICA: Uma vez no Colab, faça upload do notebook "**DLeIA_ON_Aula07_Projeto.ipynb**", leia a descrição e siga os passos iniciais para carregamento e preparação da base de dados. Em seguida, programe o que foi solicitado, fazendo uma análise crítica dos resultados.

DICA: O principal objetivo desse projeto é exercitar a construção de modelos com CNN, tanto de arquiteturas propostas e treinadas do zero quanto de transfer learning com ou sem fine-tuning.

Não se preocupe em ficar com métricas excelentes, inclusive, dependendo de quantas imagens forem utilizadas ou do número de épocas de treinamento, é normal os resultados oscilarem ou não ficarem como o esperado.

Mais importante do que chegar em boas métricas é compreender o processo como um todo! Bom trabalho!

6.3 Proposta de solução

Abaixo, temos uma proposta de solução para o problema apresentado. Essa é apenas uma forma de resolver o problema. Seus modelos podem ser diferentes e com melhores/piores resultados: o importante é praticar a construção e treinamento de CNNs, bem como o uso de TensorFlow e GPU.

DICA: Confira a solução completa no notebook "**DLeIA_ON_Aula07_ProjetoSolucao.ipynb**"

Após conferir a solução, você verá que os resultados, com e sem transfer learning, foram similares. Existes várias possíveis causas desse fenômeno:

- Falta de fine-tuning;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Modelo escolhido não ser adequado ao problema;
- Desbalanceamento dos dados;
- Métrica de desempenho não adequada para dados desbalanceados;
- Entre outras.

O que vale a pena destacar são os gráficos da Figura 29, que mostram a evolução das métricas de acurácia e perda (loss) ao longo das épocas, tanto para treino quanto para a validação:

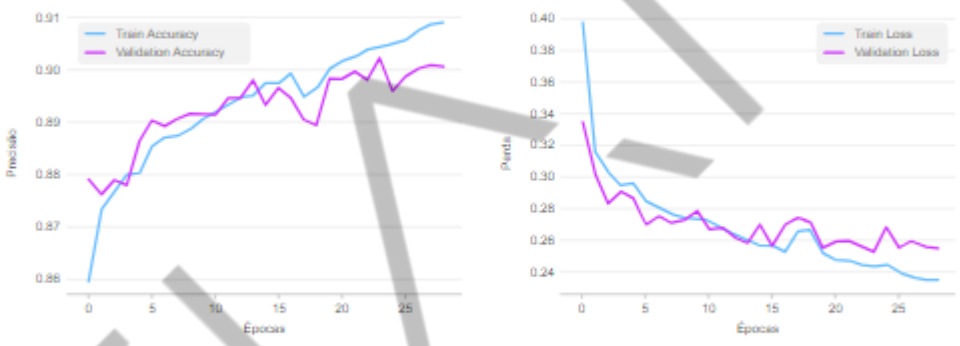


Figura 29 – Curvas de Precisão e Perda (Loss)
Fonte: Elaborado pelo autor (2024)

Como a performance (precisão) do modelo no treinamento superou significativamente a performance na validação, pode ser um sinal de overfitting, mesmo com poucas épocas.

Esse exemplo mostra que treinar redes neurais profundas é uma arte, e que o treinamento adequado nem sempre é óbvio, exigindo muita experimentação.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

BISSOTO, Alceu, et al. **Skin lesion synthesis with generative adversarial networks**. In: OR 2.0 Context-Aware Operating Theaters, Computer Assisted Robotic Endoscopy, Clinical Image-Based Procedures, and Skin Image Analysis: First International Workshop, OR 2.0 2018, 5th International Workshop, CARE 2018, 7th International Workshop, CLIP 2018, Third International Workshop, ISIC 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16 and 20, 2018, Proceedings 5. Springer International Publishing, 2018. p. 294-302.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. **Imagenet classification with deep convolutional neural networks**. Advances in neural information processing systems, 2012, 25.

TAN, Mingxing; LE, Quoc. **Efficientnet:** Rethinking model scaling for convolutional neural networks. In: International conference on machine learning. PMLR, 2019. p. 6105-6114.

VALLE, Eduardo, et al. **Data, depth, and design:** Learning reliable models for skin lesion analysis. Neurocomputing, 2020, 383: 303-313.

VASILEV, Ivan, et al. **Python Deep Learning:** Exploring deep learning techniques and neural network architectures with Pytorch, Keras, and TensorFlow. Packt Publishing Ltd, 2019.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

Termo	**Explicação**
CNNs	Convolutional neural network (Redes Neurais Convolucionais)
ELU	Exponential Linear Unit (Unidade Linear Exponencial)
FC	Fully Connected (Totalmente conectadas)
ReLU	Rectified Linear Unit (Unidade Linear Retificada)
RGB	Red, Green, Blue (Vermelho, Verde, Azul)
SIFT	Scale-Invariant Feature Transform (Transformada de Features Invariante à Escala)
SURF	Speeded Up Robust Features (Features Robustos e Acelerados)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com